矩阵键盘 / 矩阵键盘密码锁

对应江协科技《51单片机入门教程-2020版》P15《[6-1]矩阵键盘》、P16《[6-2]矩阵键盘密码锁》· 本章目标：理解矩阵省IO、会列扫描读键码、用 password*10+key%10 做4位密码锁 · 16 题分类：一、概念填空（1-5）；二、判断改错（6-9）；三、计算（10-12）；四、应用（13-16）

一、概念填空（1–5）

1. 4×4=16 个键接成行列矩阵，只占_____根 IO（4 行 + 4 列）；若用独立按键每个键一根 IO，则要_____根。

答：8；16。

思路：方法/口诀："行列交叉放按键，行+列就是线数"。**步骤：**① 4 行 + 4 列 = 8；② 独立按键 16 根；③ 省一半 IO。

常见错法：按 16×8 算；或忘了行列两端都要接 IO。

解析：矩阵省 IO 的代价是必须扫描，且耗 CPU；1080P 屏接成矩阵也是同理。

2. 本板行线（读回端）接 P1.7/P1.6/P1.5/P1.4，列线（送低电平端）接_____；扫描时某一时刻只让一列输出_____电平，再读行线判断交叉键是否按下。

答：P1.3/P1.2/P1.1/P1.0；低（0）。

思路：方法/口诀："列送0，读行==0"。**步骤：**① 一列拉低；② 该列所有键一端变0；③ 哪根行线读回0即交点按下。

常见错法：让列线输出高电平去检测；51弱上拉口输出1驱动弱、按键接地只能检测低电平。

解析：51 的 P1/P2/P3 是弱上拉准双向口，写1后外部可强行拉低被读成0，所以按键一端接地、一律检测低电平。

3. 本课程为什么用逐列扫描而不用逐行扫描？因为行扫描要翻转_____口，而它同时接了_____，会让蜂鸣器自鸣关不掉。

答：P1.5；蜂鸣器（BZ）驱动端。

思路：方法/口诀："P1.5 是蜂鸣器，别去翻转它"。**步骤：**① 行扫描要把行线逐列翻转；② P1.5 复用为 BZ；③ 改走列扫描绕开。

常见错法：不看原理图就按常规写行扫描，结果蜂鸣器一直响。

解析：这是 IO 复用冲突的典型坑，驱动代码封装好后就别乱动。

4. 密码锁键位定义：S1~S9 = 数字 1~9，S10 = 数字_____，S11 = _____，S12 = 取消，S13~S16 不用；老师示例正确密码 =_____。

> **答：**0；确认；2345。
>
> **思路：方法/口诀：**"1~9 对应，S10 是 0，S11 确认 S12 取消"。**步骤：**① 0 被"无键返回0"占用，所以用 S10 代表数字0；② 确认比对；③ 取消清场。
>
> **常见错法：**把 S10 当成密码位数 10。
>
> **解析：**正确密码用户可自定义，老师示例用 2345。

5. 每按一位数字，密码累积公式是 password = password*10 + _____；其中 key%10 的作用是把键码 10 折成数字_____。

> **答：**key%10；0。
>
> **思路：方法/口诀：**"先×10 左移，再加当前位"。**步骤：**① ×10 把已有数字左移一位；② +key%10 加上当前位；③ key=10 时 10%10=0。
>
> **常见错法：**写成 password=password+key（直接相加），结果不是逐位拼接。
>
> **解析：**复合赋值 password*=10、password+=key%10 等价。要用 count 拦住4位，否则超 65535 溢出。

二、判断改错（6–9）

6. 判断：读键时直接翻转行线做逐行扫描最自然。

> **答：**错误。改正：行扫描要翻转 P1.5，而 P1.5 同时是蜂鸣器 BZ 驱动端，会让蜂鸣器自鸣关不掉；本板改用逐列扫描——列线 P1.3~P1.0 依次送 0、读行线 P1.7~P1.4。
>
> **思路：方法/口诀：**"行扫描踩 P1.5 蜂鸣器，列扫描绕开"。**步骤：**① 先查 IO 复用；② 选列扫描方向。
>
> **常见错法：**蜂鸣器一直响，还以为程序坏了。
>
> **解析：**这是本章最具特色的坑，原理图上 P1.5=BZ 一定要看。

7. 判断：MatrixKey 检测到按下后，不做消抖、也不等松手，直接返回键码。

> **答：**错误。改正：要沿用消抖模板——检测到按下后 Delay(20) → while(按下); 等松手 → Delay(20)，再返回键码；否则一次机械抖动会被识别成多次连触发。
>
> **思路：方法/口诀：**"按20、等松手、再20、返回键码"。**步骤：**① 消抖确认；② 空转等松手；③ 松手消抖；④ return key。
>
> **常见错法：**省略 while(按下); 结果按住不放连续吐键码。
>
> **解析：**按住不放时 MatrixKey() 会阻塞在子函数里等松手，以后学定时器/中断可改非阻塞扫描。

8. 判断：密码累积直接写 password = password + key; 就行。

答：错误。改正：要写 password = password*10 + key%10;，先把已有数字×10左移一位再加当前位；同时要用 count 变量拦住4位，否则 password 超 65535 会溢出乱显。

思路：方法/口诀："×10 移位，%10 折 0，count 拦 4 位"。**步骤：**① password*=10；② +=key%10；③ count++；④ count≥4 不再接收。

常见错法：不加 count，连按5个键就溢出。

解析：不用6位密码是因为 unsigned int 装不下 999999，且 LCD_ShowNumber 只支持 0~65535。

9. 判断：解锁成功直接写 LCD_ShowString(1,1,"OK"); 就行。

答：错误。改正：上次若显过 "ERR"，末尾残留的 R 不会被覆盖，会看到 "OKR"；要写成 "OK "（后面补一个空格）才能把旧字符盖掉。确认后无论对错都要 password=0、count=0、再刷新 LCD。

思路：方法/口诀："OK 后补空格，清场必刷新"。**步骤：**① 写"OK "；② password=0；③ count=0；④ 再调显示刷新。

常见错法：清了变量却忘了刷 LCD，屏幕还留着旧数字。

解析：取消键 S12 不比对，直接清 password/count 并刷新。

三、计算（10–12）

10. 键码反推（列扫描映射）：(1) 列线 P1.3=0 时，读行线 P1.5==0，对应哪个 S 键？(2) 列线 P1.0=0 时，读行线 P1.4==0，对应哪个 S 键？

答：(1) P1.3=0 这一列：P1.7→S1、P1.6→S5、**P1.5→S9**、P1.4→S13；(2) P1.0=0 这一列：P1.7→S4、P1.6→S8、P1.5→S12、**P1.4→S16**。

思路：方法/口诀："列定列号，行定行号，S = 列+行×4"。**步骤：**① P1.3 列是第1列（S1/S5/S9/S13）；② P1.5 行是第3行 → S9；③ P1.0 列是第4列、P1.4 行是第4行 → S16。

常见错法：把列和行号加反；或把 S10 所在列记错（S10 在 P1.2=0 列、P1.5 行）。

解析：四列依次：P1.3=0→S1/S5/S9/S13；P1.2=0→S2/S6/S10/S14；P1.1=0→S3/S7/S11/S15；P1.0=0→S4/S8/S12/S16。

11. password 累积：初始 password=0、count=0。依次按下 S2、S3、S4、S5（即数字 2、3、4、5），每按一位执行 password = password*10 + key%10、count++。问：按完4位后 password=？是否等于正确密码 2345？

答：0→2→23→234→**2345**；按完正好 = **2345**，解锁成功。

思路：方法/口诀："×10 移位，逐位落位"。**步骤：**① 0×10+2=2；② 2×10+3=23；③ 23×10+4=234；④ 234×10+5=2345。

常见错法：忘×10，结果累加成 2+3+4+5=14。

解析：这就是"左移拼接"思路，key%10 把 S10 折成 0、S1~S9 折成本身。

12. 容量/省 IO：16 键矩阵用 8 根 IO，比独立按键 16 根省了几根？4 位密码用 unsigned int（0~65535）够不够？为什么课程不用 6 位密码？

答：省 16−8 = **8 根**；4 位密码**够**；不用 6 位是因为：① 6 位最大 999999 超过 unsigned int 上限 65535 装不下；② LCD_ShowNumber 只支持 0~65535、长度 1~5。

思路：方法/口诀："省一半 IO，密码位宽看类型上限"。**步骤：**① 16-8=8；② 65535 是 5 位数；③ 6 位溢出。

常见错法：以为 unsigned int 能装百万级。

解析：这也是为什么 length 参数最长只到 5。

四、应用（13–16）

13. 读程找错：下面这段 MatrixKey 扫描片段想读第一列，有两处明显问题，请指出。

```
P1 = 0xF7;
if(P1_7 == 0)
{
    key = 1;
}
```

答：两处问题——① 命中后没有消抖（Delay(20)）、没有等松手（while(按下);）、没有松手消抖，会连触发；② 只扫了 P1.3=0 这一列就结束，没有依次扫 P1.2/P1.1/P1.0 三列，S2~S16 永远读不到。正确骨架是四列依次送 0、每列里判断四根行线、命中走消抖+等松手模板再 return key。

```
P1 = 0xF7;            /* P1.3=0 */
if(P1_7 == 0){ Delay(20); while(P1_7==0); Delay(20); key=1; }
if(P1_6 == 0){ Delay(20); while(P1_6==0); Delay(20); key=5; }
/* …P1.2/P1.1/P1.0 三列同理，最后 return key; */
```

思路：方法/口诀："四列扫全，命中走消抖模板"。**步骤：**① P1=0xF7/0xFB/0xFD/0xFE 轮流；② 每列查 P1_7~P1_4；③ 命中 Delay(20)→while→Delay(20)。

常见错法：只写一列；或命中后直接 return 不走等松手。

解析：代码为自写示例（非原课程源码）。无键时 key 保持 0，由 return 带回 main。

14. 编程（调用）：MatrixKey() 返回键码后，main 里把键码显示到 LCD 第二行。写出关键调用语句，并说明为什么要套一个 if(key)。

答：

```
key = MatrixKey();
if(key)                        /* 即 if(key != 0)，非0才显示 */
{
    LCD_ShowNumber(2, 1, key, 2);
}
```

不加 if(key) 的后果：主循环每秒反复调 MatrixKey，没键时 key=0 会立刻把上次显示的键码刷成 0，肉眼看到数字抖一下就没。

思路：方法/口诀："有键才刷新，无键保留上次"。**步骤：**① 调 MatrixKey；② if(key) 判断；③ ShowNumber 两位宽。

常见错法：省略 if(key)，数字闪一下消失。

解析：if 里只放变量时"非0即真"，要显式判断等于0才写 ==0。代码为自写示例（非原课程源码）。

15. 编程（确认键）：当确认键 S11 按下时，比对 password 是否等于 2345，在第一行末尾显示 "OK " 或 "ERR"，然后清场准备下一次输入。写出该分支代码。

答：

```
if(key == 11)                  /* S11 确认 */
{
    if(password == 2345)
        LCD_ShowString(1, 8, "OK ");    /* 补空格覆盖 ERR 残留 */
    else
        LCD_ShowString(1, 8, "ERR");
    password = 0;              /* 清场 */
    count = 0;
    /* 再刷新 LCD 上 password 的显示 */
}
```

思路：方法/口诀："比对、反馈、清场、刷新"。**步骤：**① key==11；② ==2345 显"OK "否则"ERR"；③ password=0、count=0；④ 刷新显示。

常见错法："OK" 不补空格；或清了变量忘刷屏。

解析：取消键 S12 分支则不比对，直接 password=0、count=0、刷新。代码为自写示例（非原课程源码）。

16. 逻辑题：取消键 S12 该做什么？为什么清完 password/count 之后还要再调一次 LCD 刷新显示？

答：S12 不做比对，直接执行 password=0、count=0，并刷新 LCD 显示（把屏上已输入的数字擦掉）。原因：变量清零只是改了内存里的数，LCD 屏幕上仍留着上次显示的数字；不再刷一次，用户会看到"逻辑上清了、屏幕上还留着"的不一致。

思路：方法/口诀："取消=不比对，清场+同步屏幕"。**步骤：**① password=0；② count=0；③ 重画显示。

常见错法：只清变量不刷屏。

解析：这是"状态变量"和"显示"必须同步刷新的通用原则，确认分支同样适用。
